

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C 도서관 검색 개념정리	date

2026.06.08

C 도서관 검색 프로그램 개념정리

핵심

도서관 검색 프로그램은 책 정보를 저장하고, 정렬하고, 빠르게 찾는 과정을 연습하는 프로그램이다. 이 과제는 C에서 구조체, 배열, 문자열 처리, 정렬 알고리즘, 이진 탐색을 한 번에 연결해서 이해하기 위해 필요하다.

왜 필요한가

프로그래밍은 단순히 문법을 외우는 것이 아니라 데이터를 어떻게 저장하고, 어떤 기준으로 처리하고, 어떻게 빠르게 찾을지 결정하는 과정이다. 도서관 검색 프로그램은 실제 프로그램에서 자주 나오는 "데이터 목록 관리"의 기본 형태이다.

예를 들어 도서관에는 책이 많다. 사용자가 책 제목을 입력했을 때 컴퓨터는 책 목록에서 맞는 책을 찾아야 한다. 책이 5권이면 하나씩 찾아도 괜찮지만, 책이 1만 권이면 처음부터 끝까지 찾는 방식은 느려진다. 그래서 정렬과 이진 탐색이 필요하다.

Book 구조체

C에서는 책 한 권의 정보를 구조체로 묶을 수 있다.

```
typedef struct {
    int id;
    char title[100];
    char author[50];
} Book;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

책 번호, 제목, 저자를 따로따로 배열로 관리하면 헷갈리기 쉽다. 구조체를 사용하면 책 한 권의 정보를 하나의 단위로 묶을 수 있다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C 도서관 검색 개념정리	date

구조체가 필요한 이유

구조체는 관련 있는 데이터를 하나로 묶기 위해 필요하다. 책 제목 배열, 저자 배열, 번호 배열을 따로 만들면 같은 인덱스를 계속 맞춰야 한다. 이 방식은 실수하기 쉽다.

```
int ids[100];
char titles[100][100];
char authors[100][50];
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

위 방식보다 아래 방식이 더 자연스럽다.

```
Book books[100];
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

정리 구조체를 사용하면 데이터의 의미가 분명해지고, 함수에 책 정보를 넘기기도 쉬워진다.

문자열 처리

C에서는 문자열을 `char 배열`로 저장한다.

```
char title[100] = "C Programming";
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

C에서 문자열 비교는 `==`가 아니라 `strcmp()`를 사용한다.

```
if (strcmp(title, "C Programming") == 0) {
    printf("같은 제목\n");
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

왜 strcmp가 필요한가

C의 문자열은 문자들이 들어 있는 배열이다. 배열 이름은 실제 문자열 내용 전체가 아니라 시작 주소처럼 동작한다. 그래서 `==`를 쓰면 문자열 내용이 같은지 비교하는 것이 아니라 주소를 비교하는 문제가 생길 수 있다.

정리 C에서 문자열 내용을 비교하려면 반드시 `strcmp()`를 사용해야 한다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C 도서관 검색 개념정리	date

정렬이 필요한 이유

정렬은 데이터를 일정한 기준에 따라 순서대로 배치하는 것이다.

정렬 전:

Data Structure

Algorithm

C Programming

정렬 후:

Algorithm

C Programming

Data Structure

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

정렬이 필요한 가장 중요한 이유는 이진 탐색을 사용하기 위해서이다. 이진 탐색은 가운데 값을 기준으로 왼쪽과 오른쪽을 선택하는 방식이므로, 데이터가 정렬되어 있지 않으면 어느 쪽을 탐색해야 할지 판단할 수 없다.

정렬 기준과 검색 기준

정렬 기준과 검색 기준은 같아야 한다.

- 제목으로 검색하려면 제목순 정렬
- 번호로 검색하려면 번호순 정렬
- 저자로 검색하려면 저자명순 정렬

주의 제목순으로 정렬해놓고 책 번호로 이진 탐색하면 올바른 결과가 나오지 않는다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C 도서관 검색 개념정리	date

버블 정렬 예시

```
void sortByTitle(Book books[], int count) {
    int i, j;

    for (i = 0; i < count - 1; i++) {
        for (j = 0; j < count - 1 - i; j++) {
            if (strcmp(books[j].title, books[j + 1].title) > 0) {
                Book temp = books[j];
                books[j] = books[j + 1];
                books[j + 1] = temp;
            }
        }
    }
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`strcmp()` 결과가 0보다 크면 앞의 제목이 사전순으로 뒤에 있다는 뜻이다. 이때 두 책의 위치를 바꾼다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C 도서관 검색 개념정리	date

이진 탐색이 필요한 이유

순차 탐색은 처음부터 끝까지 하나씩 확인한다. 구현은 쉽지만 데이터가 많으면 느리다.

이진 탐색은 정렬된 데이터에서 가운데 값을 확인하고, 찾는 값이 가운데보다 작으면 왼쪽, 크면 오른쪽만 탐색한다. 한번 비교할 때마다 검색 범위가 절반으로 줄어든다.

```
int binarySearchByTitle(Book books[], int count, char keyword[]) {
    int left = 0;
    int right = count - 1;

    while (left <= right) {
        int mid = (left + right) / 2;
        int result = strcmp(books[mid].title, keyword);

        if (result == 0) {
            return mid;
        } else if (result < 0) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return -1;
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

결과 찾으면 배열 인덱스를 반환하고, 못 찾으면 -1을 반환한다.

순차 탐색과 이진 탐색 비교

순차 탐색	정렬이 필요 없고 구현이 쉽다.
이진 탐색	정렬이 필요하지만 데이터가 많을수록 빠르다.

정리 데이터가 적으면 순차 탐색도 괜찮지만, 데이터가 많아질 가능성이 있으면 정렬 후 이진 탐색을 사용하는 것이 좋다.

최종 정리

도서관 검색 프로그램은 C 문법을 실제 문제에 적용하는 연습이다. 구조체는 책 정보를 묶기 위해 필요하고, 문자열 함수는 C에서 문자열을 정확히 비교하기 위해 필요하다. 정렬은 데이터를 기준에 맞게 정리하고 이진 탐색을 가능하게 하기 위해 필요하다. 이진 탐색은 많은 데이터 중 원하는 값을 빠르게 찾기 위해 필요하다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C 도서관 검색 개념정리	date

확인 질문

- 구조체는 왜 필요한가?
- C에서 문자열 비교에 `strcmp()`를 쓰는 이유는 무엇인가?
- 정렬 기준과 검색 기준이 같아야 하는 이유는 무엇인가?
- 이진 탐색이 순차 탐색보다 유리한 상황은 언제인가?

어디에 사용하는가

도서관 검색 프로그램 구조는 실제로 많은 프로그램에서 사용된다. 이름만 도서관일 뿐, 원리는 "목록에서 원하는 데이터를 찾는 프로그램"이다.

사용 예시:

- 도서관에서 책 제목으로 책 찾기
- 쇼핑몰에서 상품명으로 상품 찾기
- 학생 명단에서 학번으로 학생 찾기
- 연락처 목록에서 이름으로 전화번호 찾기
- 파일 목록에서 특정 파일명 찾기

어떻게 사용하는가

먼저 데이터를 구조체 배열에 저장한다. 그다음 검색 기준을 정한다. 제목으로 찾을지, 번호로 찾을지 결정한 뒤 그 기준에 맞게 정렬한다. 마지막으로 이진 탐색을 사용해 원하는 데이터를 찾는다.

예시 흐름:

```
Book 구조체 만들기
-> Book 배열에 책 저장
-> 제목순 정렬
-> 사용자에게 제목 입력받기
-> 이진 탐색으로 찾기
-> 결과 출력
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

비유

책장이 제목순으로 정리되어 있으면 원하는 책을 빨리 찾을 수 있다. 하지만 책이 아무렇게나 꽂혀 있으면 처음부터 끝까지 하나씩 봐야 한다. 이진 탐색은 책장이 이미 정리되어 있을 때 가능한 빠른 찾기 방법이다.